

# Destructuring in JavaScript — Beginner to Advanced

## 1. What is Destructuring?

**Destructuring** is an ES6 feature in JavaScript that allows us to **extract values** from **Arrays or Properties from Objects** and **store them directly into variables**. 😊

### Without Destructuring

```
const student = {  
  name: "Kabir",  
  age: 25,  
  city: "Pune"  
};  
  
//If you want to access the name, age, and city, then in the following way  
//we do  
  
const name = student.name;  
const age = student.age;  
const city = student.city;  
  
console.log(name);  
console.log(age);  
console.log(city);  
  
Here, we access every property using:  
student.name  
student.age  
student.city
```

## With Destructuring

```
const student = {  
  name: "Kabir",  
  age: 25,  
  city: "Pune"  
};  
  
const { name, age, city } = student;  
  
console.log(name);  
console.log(age);  
console.log(city);
```

Destructuring is a technique used to unpack values from arrays or properties from objects into separate variables.

---

## 2. Why Do We Need Destructuring? 🤔

*Object destructuring allows us to extract required properties from an Object & elements from an Array and use them directly as variables, making the code shorter, cleaner, and easier to read.*

1. Cleaner code
  2. Less repetitive code
  3. Easy access to object properties and Array Elements
  4. Makes code easier to read
-

## 3. Types of Destructuring

There are mainly two types:

### 1. Array Destructuring

### 2. Object Destructuring

---

## 4. Array Destructuring

Suppose we have:

```
const numbers = [10, 20, 30];
```

Normally, if you want to access elements, then in the following way we access them:

```
const a = numbers[0];  
const b = numbers[1];  
const c = numbers[2];
```

Using destructuring:

```
const [a, b, c] = numbers;  
  
console.log(a); // 10  
console.log(b); // 20
```

```
console.log(c); // 30
```

### How does it work?

The position matters.

Array

| Index | Value |
|-------|-------|
| 0     | 10    |
| 1     | 20    |
| 2     | 30    |

↓ Destructuring

[a, b, c]

a → 10

b → 20

c → 30

---

## 5. Array Destructuring is Position-Based

This is extremely important.

```
const numbers = [10, 20, 30];  
  
const [x, y, z] = numbers;
```

Result:

x = 10

y = 20

z = 30

Because:

x → index 0

y → index 1

z → index 2

Unlike objects, array destructuring does **not care about property names**.

---

## 6. Skipping Array Values

Sometimes we don't need every value.

```
const numbers = [10, 20, 30];
```

```
const [a, , c] = numbers;
```

```
console.log(a); // 10
```

```
console.log(c); // 30
```

The second value is skipped.

10   20   30

↓

a

↓

c

Another example:

```
const data = [100, 200, 300, 400];

const [, second, , fourth] = data;

console.log(second); // 200
console.log(fourth); // 400
```

---

## 7. Default Values in Array Destructuring

Suppose:

```
const numbers = [10, 20];

const [a, b, c] = numbers;

console.log(c);

Output:
undefined
```

We can provide a default value:

```
const [a, b, c = 30] = numbers;

console.log(c); // 30
```

### Important

Default value is used only when the value is **undefined**.

```
const [a = 100] = [undefined];
```

```
console.log(a); // 100
```

But:

```
const [a = 100] = [null];  
  
console.log(a); // null
```

*Default value does not replace null.*

---

## 8. Rest Operator with Array Destructuring

The `...` syntax can collect the remaining values.

```
const numbers = [10, 20, 30, 40, 50];  
  
const [first, second, ...remaining] = numbers;  
  
console.log(first);      // 10  
console.log(second);     // 20  
console.log(remaining);  // [30, 40, 50]
```

### Important

The rest variable must be the **last element**.

Correct:

```
const [a, b, ...rest] = numbers;
```

Incorrect:

```
const [...rest, a] = numbers;
```

---

## 9. Swapping Variables Using Destructuring

One of the most useful features.

### Traditional Approach

```
let a = 10;
let b = 20;

let temp = a;
a = b;
b = temp;

console.log(a); // 20
console.log(b); // 10
```

### Using Destructuring

```
let a = 10;
let b = 20;
```

```
[a, b] = [b, a];
```

```
console.log(a); // 20
```



```
console.log(b); // 10
```

This is called **destructuring assignment**.

---

## 10. Nested Array Destructuring

Arrays can contain arrays.

```
const numbers = [10, [20, 30]];

const [a, [b, c]] = numbers;

console.log(a); // 10
console.log(b); // 20
console.log(c); // 30
```

---

## 11. Object Destructuring

Now let's understand object destructuring.

```
const student = {
  name: "Kabir",
  age: 25,
  city: "Pune"
};
```

Without destructuring:

```
const name = student.name;  
const age = student.age;  
const city = student.city;
```

With destructuring:

```
const { name, age, city } = student;
```

---

## 12. Object Destructuring is Name-Based

This is an important difference.

```
const student = {  
  name: "Kabir",  
  age: 25  
};  
  
const { age, name } = student;  
  
console.log(name); // Kabir  
console.log(age);  // 25
```

The order doesn't matter.

Why?

Because object destructuring works using **property key names**.

---

## 13. Renaming Variables During Destructuring

Suppose:

```
const student = {  
  name: "Kabir",  
  age: 25  
};
```

We don't want the variable to be called `name`.

We can rename it:

```
const { name: studentName, age: studentAge } = student;  
  
console.log(studentName); // Kabir  
console.log(studentAge);  // 25
```

Syntax:

```
const { propertyName: variableName } = object;
```

### Important

This:

```
const { name: studentName } = student;
```

does **not** mean:

```
name = studentName
```

It means:

object property → variable

name → studentName

---

## 14. Default Values in Object Destructuring

```
const user = {  
  name: "Kabir"  
};  
  
const { name, age = 25 } = user;  
  
console.log(name); // Kabir  
console.log(age);  // 25
```

Since `age` does not exist:

```
age = 25
```

---

## 15. Rename + Default Value

We can combine both.

```
const user = {
  name: "Kabir"
};

const {
  name: userName,
  age: userAge = 25
} = user;

console.log(userName); // Kabir
console.log(userAge);  // 25
```

---

## 16. Rest Operator with Object Destructuring

Suppose:

```
const user = {
  name: "Kabir",
  age: 25,
  city: "Pune",
  role: "Developer"
};
```

We want `name` separately and the remaining properties together.

```
const { name, ...otherDetails } = user;
```

```
console.log(name);  
console.log(otherDetails);
```

Output:

Kabir

```
{  
  age: 25,  
  city: "Pune",  
  role: "Developer"  
}
```

## Important

The rest property collects the **remaining properties**.

---

# 17. Nested Object Destructuring

Consider:

```
const user = {  
  name: "Kabir",  
  address: {  
    city: "Pune",  
    country: "India"  
  }  
};
```

We can destructure nested properties:

```
const {
  name,
  address: {
    city,
    country
  }
} = user;

console.log(name);
console.log(city);
console.log(country);
```

Output:

Kabir  
Pune  
India

---

## 18. Nested Object + Renaming

```
const user = {
  name: "Kabir",
  address: {
    city: "Pune"
  }
};

const {
  name: userName,
  address: {
    city: userCity
  }
} = user;
```

```
console.log(userName);  
console.log(userCity);
```

---

## 19. Mixed Destructuring

Objects can contain arrays and arrays can contain objects.

Example:

```
const user = {  
  name: "Kabir",  
  skills: ["JavaScript", "React", "Node"]  
};
```

We can do:

```
const {  
  name,  
  skills: [skill1, skill2, skill3]  
} = user;
```

```
console.log(name);  
console.log(skill1);  
console.log(skill2);  
console.log(skill3);
```

Output:

```
Kabir  
JavaScript  
React  
Node
```



---

## 20. Array Containing Objects

Consider:

```
const users = [  
  {  
    name: "Kabir",  
    age: 25  
  },  
  {  
    name: "Rahul",  
    age: 26  
  }  
];
```

We can destructure:

```
const [  
  { name: name1, age: age1 },  
  { name: name2, age: age2 }  
] = users;  
  
console.log(name1);  
console.log(age1);  
  
console.log(name2);  
console.log(age2);
```

---

## 21. Destructuring in Function Parameters

This is extremely important in modern JavaScript and React.

Consider:

```
function displayUser(user) {  
  console.log(user.name);  
  console.log(user.age);  
}
```

We can destructure directly inside the parameters:

```
function displayUser({ name, age }) {  
  console.log(name);  
  console.log(age);  
}  
  
displayUser({  
  name: "Kabir",  
  age: 25  
});
```

**Why is this useful?**

Instead of:

```
user.name  
user.age  
user.email
```

we directly get:

```
name  
age  
email
```

---

## 22. Function Parameter + Renaming

```
function displayUser({ name: userName, age: userAge }) {  
    console.log(userName);  
    console.log(userAge);  
}  
  
displayUser({  
    name: "Kabir",  
    age: 25  
});
```

---

## 23. Function Parameter Default Value

```
function displayUser({ name, age = 18 }) {  
    console.log(name, age);  
}  
  
displayUser({  
    name: "Kabir"  
});
```

**Output :**  
Kabir 18

---

## 24. Destructuring Function Return Value

A function can return an array:

```
function getUser() {  
    return ["Kabir", 25];  
}  
  
const [name, age] = getUser();  
  
console.log(name);  
console.log(age);
```

This is very useful when a function returns multiple values.

---

## 25. Destructuring Object Returned by Function

```
function getUser() {  
    return {  
        name: "Kabir",  
        age: 25  
    };  
}  
  
const { name, age } = getUser();  
  
console.log(name);  
console.log(age);
```

---

## 26. Destructuring API Response

Suppose an API returns:

```
const response = {  
  status: 200,  
  data: {  
    name: "Kabir",  
    age: 25  
  }  
};
```

Instead of:

```
console.log(response.data.name);  
console.log(response.data.age);
```

We can write:

```
const {  
  data: {  
    name,  
    age  
  }  
} = response;  
  
console.log(name);  
console.log(age);
```

---

## 27. Destructuring with let and const

We can use:

```
const [a, b] = [10, 20];
```

or:

```
let [a, b] = [10, 20];
```

But there is an important case.

### Existing Variables

```
let a = 10;  
let b = 20;  
  
[a, b] = [b, a];  
  
Notice:  
[a, b] = ...
```

There is no `const` or `let`.

This is called **Destructuring assignment**

---

## 28. Deep Nested Destructuring

Consider:

```
const company = {  
  name: "VK AI TECH",  
  owner: {  
    name: "Kabir",  
    address: {  
      city: "Pune",  
      country: "India"  
    }  
  }  
};
```

We can extract everything:

```
const {  
  name: companyName,  
  owner: {  
    name: ownerName,  
    address: {  
      city,  
      country  
    }  
  }  
} = company;  
  
console.log(companyName);  
console.log(ownerName);  
console.log(city);  
console.log(country);
```

This is called **deep/nested destructuring**.

---

## 29. Destructuring with Dynamic Keys

```
const key = "age";

const user = {
  name: "Kabir",
  age: 25
};

const { [key]: userValue } = user;

console.log(userValue); // 25
```

This is useful when the property name is stored dynamically in a variable.

---

## 30. Destructuring and Function Safety

Consider:

```
function displayUser({ name }) {
  console.log(name);
}
```

```
Calling:
displayUser();
```

causes an error.

We can provide a default object:



```
function displayUser({ name } = {}) {  
  console.log(name);  
}
```

Now:

```
displayUser();
```

does not throw an error.

---

## 31. Destructuring with for...of

Suppose:

```
const users = [  
  { name: "Kabir", age: 25 },  
  { name: "Rahul", age: 26 }  
];
```

Instead of:

```
for (const user of users) {  
  console.log(user.name);  
}
```

we can write:

```
for (const { name, age } of users) {  
  console.log(name, age);  
}
```

This is very clean.

---

## 32. Destructuring with map ( )

Consider:

```
const users = [  
  { name: "Kabir", age: 25 },  
  { name: "Rahul", age: 26 }  
];
```

### Without Destructuring

```
const names = users.map(user => user.name);
```

- `map()` takes each object one by one.
- `user` represents the complete object.
- `user.name` gets the `name` property.
- Result: ["Kabir", "Rahul"]

### With Destructuring

```
const names = users.map(({ name }) => name);
```

Here:

```
({ name })
```

directly extracts the `name` property from each object.

```
["Kabir", "Rahul"]
```

## Simple Meaning

```
users.map(({ name }) => name);
```

means:

**Go through each user, extract the `name`, and create a new array containing those names.**

---

## Real-World Use Case — Function Options

Instead of creating a function with many parameters:

```
function createUser(name, age, city, role) {  
}
```

we can use an options object:

```
function createUser({  
  name,  
  age,  
  city,  
  role  
) {  
  console.log(name);  
  console.log(age);  
  console.log(city);  
}
```

```
    console.log(role);  
}
```

Call:

```
createUser({  
  name: "Kabir",  
  age: 25,  
  city: "Pune",  
  role: "Developer"  
});
```

This approach becomes very useful when a function has many optional parameters.

---

## Interview Questions

### Q1. What is destructuring?

Destructuring is an ES6 feature that allows us to extract values from arrays and properties from objects into variables.

---

### Q2. What is the difference between array and object destructuring?

Array destructuring is position-based, whereas object destructuring is property-name-based.

---

### Q3. Can we rename variables during object destructuring?

Yes.

```
const { name: userName } = user;
```

---

#### Q4. Can we provide default values?

Yes.

```
const { age = 18 } = user;
```

and:

```
const [age = 18] = [];
```

---

#### Q5. Can we use rest with destructuring?

Yes.

```
const [first, ...rest] = numbers;
```

and:

```
const { name, ...others } = user;
```

---

#### Q6. Can destructuring be nested?

Yes.

```
const {  
  address: {  
    city  
  }  
} = user;
```

---

### Q7. Is destructuring a copy?

**Destructuring itself does not perform a deep copy.**

For primitive values:

```
const user = {  
  age: 25  
};  
  
const { age } = user;
```

**age** receives the primitive value.

For objects:

```
const user = {  
  address: {  
    city: "Pune"  
  }  
};  
  
const { address } = user;
```

**address** refers to the same nested object.

Therefore:

```
address.city = "Mumbai";
```

also changes:

```
user.address.city
```

because both reference the same object.

---

## Important Concept — Destructuring Does Not Mean Cloning

```
const user = {  
  name: "Kabir",  
  address: {  
    city: "Pune"  
  }  
};  
  
const { address } = user;  
  
address.city = "Mumbai";  
  
console.log(user.address.city);
```

Output:  
Mumbai

---

## Advanced Example

```
const response = {
  status: 200,
  data: {
    user: {
      name: "Kabir",
      skills: ["JavaScript", "React", "Node"]
    }
  }
};
```

```
const {
  status,
  data: {
    user: {
      name,
      skills: [firstSkill, ...otherSkills]
    }
  }
} = response;

console.log(status);
console.log(name);
console.log(firstSkill);
console.log(otherSkills);
```

```
200
Kabir
JavaScript
["React", "Node"]
```

This single example combines:

- Object destructuring



- Nested destructuring
  - Array destructuring
  - Rest operator
  - Renaming concepts
  - API response handling
-